

What will my scanner do? (list exact features)

- Scan the port of a given IP address
- Output whether it is open or closed based on the response

What will decide whether a port is opened or closed?

- If a port gives a response in a given time then it will be deemed open otherwise closed

What won't it do? (scope limits)

- Scan for any potential vulnerabilities
- Detect the services available on the ports
- Detect the OS

What are the main components? (scanner engine, results storage, output formatter)

- The layer where the scanning logic resides
- The interface layer where the command line logic will reside and call upon the scanner logic

What do I need to learn? (list unknowns)

- How to actually scan ports in Python (and Golang in the future)
- How to create a simple command line tool in Python (and Golang in the future)
- What is a basic ping, a SYN packet and an ARP request

What will be the scanner logic

1. Identifying host based on the given IP

The program will send packets to the given IP addresses over the network and deem it online if it responds.

The program this is inspired by (nmap) does this in a couple of ways:

- Sending a basic ping
- Sending simple SYN packets
- Sending an ARP request

2. Port Scanning

nmap can do this using multiple techniques:

- **TCP Connect Scan (-sT):** This is the simplest technique. Nmap completes the TCP handshake by sending a SYN packet to the target. If the port is open, the target responds with a SYN-ACK packet.
- **SYN Scan (-sS):** The most popular and stealthy scan. Nmap sends a SYN packet to the target port but does not complete the TCP handshake. If the port is open, the target will respond with a SYN-ACK packet.

There are other techniques of course but these two will be the priority for now. The one with the highest priority is the TCP one.

Overview:

Of course there are loads of other stuff nmap and potentially this little project can do if I decide to implement them but for now the focus will be this.